

Roteiro de Atividade

Filtros de Aguçamento, Detecção de Pontos/Linhas/Bordas e LoG / DoG

Referência: Gonzalez & Woods, *Digital Image Processing*

Como combinado na última aula, o laboratório de hoje pode ser feito em seu computador em casa ou no trabalho. Envie suas dúvidas ou conversem com o professor pessoalmente. As apresentações e *notebooks* devem ser submetidos no Google Classroom.

1. Aguçamento

Leia a Seção 3.6 de Gonzalez & Woods sobre **filtros de aguçamento**: os fundamentos da diferenciação espacial e a utilização da segunda derivada para realçar detalhes de uma imagem. O Laplaciano é a soma das derivadas parciais de segunda ordem:

$$\nabla^2 f(x, y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}. \quad (1)$$

Usando diferenças centrais, obtém-se a aproximação discreta

$$\nabla^2 f(x, y) = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y). \quad (2)$$

O aguçamento é obtido combinando a imagem original com a imagem laplaciana:

$$g(x, y) = f(x, y) + c \nabla^2 f(x, y), \quad (3)$$

onde $c = -1$ quando a máscara laplaciana possui centro negativo, e $c = +1$ quando a máscara possui centro positivo.

Submissão

Prepare uma apresentação estilo PowerPoint sobre a Seção 3.6, explicando as derivadas mostradas na Figura 3.36, a obtenção das máscaras de filtragem da Figura 3.37 e o aguçamento de imagens utilizando o Laplaciano, como no Exemplo 3.15. No *notebook* Colab, reproduza os resultados da Figura 3.38. Submeta a apresentação e o *notebook* no Google Classroom.

2. Detecção de pontos e linhas

No Capítulo 10, Gonzalez & Woods retomam o uso de derivadas de primeira e segunda ordens no contexto de **segmentação**, em particular para detectar mudanças abruptas locais de intensidade. Leia a Seção 10.2, que discute a detecção de pontos, linhas e bordas. A resposta de uma máscara w com coeficientes w_k , aplicada a uma vizinhança de intensidades z_k , pode ser escrita como

$$R = \sum_{k=1}^9 w_k z_k = w_1 z_1 + \dots + w_9 z_9. \quad (4)$$

A detecção é usualmente feita comparando-se a magnitude dessa resposta com um limiar:

$$|R| \geq T. \quad (5)$$

Submissão

Prepare uma apresentação estilo PowerPoint sobre detecção de **pontos, linhas e linhas em direções específicas**. Inclua as máscaras direcionais e discuta como o sinal e a magnitude da resposta indicam a presença e a orientação de estruturas lineares. Submeta a apresentação no Google Classroom.

3. Modelos de borda

Bordas ideais e reais são discutidas na Seção 10.2.4, juntamente com os passos básicos de detecção: suavização, diferenciação e localização. Na Seção 10.2.5 aparecem técnicas baseadas em derivadas de primeira ordem, como Prewitt e Sobel. A atividade a seguir concentra-se em métodos baseados na segunda derivada, em especial o **Laplaciano da Gaussiana**, sua aproximação por **Diferença de Gaussianas** e o **algoritmo de Marr-Hildreth**, que reúne essas peças em um detector de bordas completo.

Vamos comparar aqui a máscara LoG 5×5 apresentada por Gonzalez & Woods, o LoG obtido por amostragem da função contínua e a DoG. A comparação deve considerar a convenção de sinal, os cruzamentos por zero, o tamanho do suporte do filtro, a razão entre desvios padrão, o efeito do arredondamento dos coeficientes e a equivalência entre suavizar-e-derivar e o operador LoG direto.

LoG e convenção de sinal

A **Laplaciana da Gaussiana** (LoG) combina suavização gaussiana e diferenciação de segunda ordem em um único operador. A suavização reduz a sensibilidade ao ruído, enquanto o Laplaciano realça transições locais de intensidade. Por isso, o LoG é frequentemente usado para detecção de bordas por meio dos seus cruzamentos por zero. Considerando a gaussiana sem o fator de normalização,

$$G(x, y) = e^{-\frac{x^2+y^2}{2\sigma^2}}, \quad (6)$$

sua Laplaciana pode ser escrita como

$$\nabla^2 G(x, y) = \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} e^{-\frac{x^2+y^2}{2\sigma^2}}. \quad (7)$$

Com essa convenção, o valor central é negativo. Gonzalez & Woods, entretanto, apresentam a função com o sinal invertido em algumas ilustrações, de modo que o perfil apareça com lobo central positivo e anel periférico negativo, isto é, na orientação visual usual do “chapéu mexicano”. Os próprios autores indicam que, para uso como filtro LoG sob a convenção de sinal correspondente à Equação (7), a máscara exibida deve ser multiplicada por -1 . A máscara 5×5 apresentada por Gonzalez & Woods é

$$\mathbf{M} = \begin{bmatrix} 0 & 0 & -1 & 0 & 0 \\ 0 & -1 & -2 & -1 & 0 \\ -1 & -2 & 16 & -2 & -1 \\ 0 & -1 & -2 & -1 & 0 \\ 0 & 0 & -1 & 0 & 0 \end{bmatrix}. \quad (8)$$

Ela possui centro positivo e soma exatamente nula. A versão compatível com a convenção de centro negativo da Equação (7) é $-\mathbf{M}$.

Observação. Multiplicar o filtro por -1 inverte o sinal da resposta, mas não altera a posição dos cruzamentos por zero. Portanto, a convenção de sinal deve ser mantida consistente com a interpretação da resposta e com a etapa de combinação ou visualização posterior.

Ponto de partida obrigatório: máscara de Gonzalez & Woods

Antes de implementar LoG amostrado e DoG, aplique diretamente a máscara $\mathbf{M}$ da Equação (8) a uma imagem sintética em tons de cinza. Use fundo preto e objetos claros, por exemplo, um quadrado branco e um círculo branco. Em seguida, aplique $-\mathbf{M}$ à mesma imagem. Compare as duas respostas e localize os cruzamentos por zero em ambos os casos. Esse teste deve servir como referência: ele mostra claramente que a inversão de sinal troca regiões positivas por negativas, mas preserva a geometria das bordas obtidas por cruzamentos por zero.

Observação. O livro não explicita o fator de escala usado para obter os coeficientes inteiros de $\mathbf{M}$ a partir da função LoG contínua. Portanto, trate $\mathbf{M}$ como uma aproximação discreta compacta e de soma nula, não como uma discretização única ou normalizada da função contínua.

LoG por amostragem

Depois do teste com a máscara de Gonzalez & Woods, gere filtros LoG por amostragem da função contínua. O código abaixo usa suporte $(2k+1) \times (2k+1)$, centro em (k, k) , normalização pela norma Euclidiana e arredondamento para três casas decimais.

```
from numpy import pi
import numpy as np
def LOG(k, s):
    n = 2*k + 1
    filtro = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
```

```

r2 = (i-k)**2 + (j-k)**2
filtro[i, j] = -(1 - r2/(2.*s**2)) * np.exp(-r2/(2.*s
**2)) / (pi*s**4)
filtro = np.round(filtro / np.sqrt((filtro**2).sum()), 3)
return filtro

```

A função acima produz um filtro com centro negativo, compatível com a convenção da Equação (7). Para compará-lo com a máscara **M** mostrada por Gonzalez & Woods, compare também o seu negativo.

Observação. Em suportes pequenos, como 5×5 , a função LoG pode ser truncada de modo severo. Além disso, a normalização e o arredondamento podem fazer com que a soma dos coeficientes se afaste de zero. Quando o objetivo for implementar um filtro de segunda derivada, é razoável subtrair a média dos coeficientes após a amostragem para impor soma nula.

DoG — Diferença de Gaussianas

A **Diferença de Gaussianas (DoG)** é definida como a diferença entre duas gaussianas com desvios padrão distintos:

$$DoG(x, y) = \frac{1}{2\pi\sigma_1^2} e^{-\frac{x^2+y^2}{2\sigma_1^2}} - \frac{1}{2\pi\sigma_2^2} e^{-\frac{x^2+y^2}{2\sigma_2^2}} \quad (9)$$

onde $\sigma_1 > \sigma_2$. Gonzalez & Woods observam que certos canais do sistema visual humano podem ser modelados por uma DoG com razão entre desvios padrão de aproximadamente 1,75:1. A razão 1,6:1, por sua vez, preserva as características básicas dessa observação e fornece uma boa aproximação do LoG, conforme Marr e Hildreth. A implementação recebe σ_1 e σ_2 de forma independente, de modo que a razão $\sigma_1:\sigma_2$ possa ser escolhida livremente (por exemplo 1,6:1 ou 1,75:1):

```

def DOG(k, s1, s2):      # s1 > s2 ; razao = s1/s2
    n = 2*k + 1
    filtro = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            r2 = (i-k)**2 + (j-k)**2
            filtro[i, j] = (
                np.exp(-r2/(2.*s1**2))/(2*pi*s1**2)
                - np.exp(-r2/(2.*s2**2))/(2*pi*s2**2)
            )
    filtro = np.round(filtro / np.sqrt((filtro**2).sum()), 3)
    return filtro

```

Observação. Evite fixar internamente $\sigma_1 = s\sqrt{2}$ e $\sigma_2 = s/\sqrt{2}$: isso traria a razão em 2,0:1 para qualquer s e impediria testar as razões 1,6:1 e 1,75:1 pedidas nos experimentos. Use, por exemplo, DOG(2, 1.6, 1.0) e DOG(2, 1.75, 1.0).

Para que a LoG e a DoG tenham os mesmos cruzamentos por zero, o desvio padrão σ da LoG deve ser escolhido a partir dos desvios padrão σ_1 e σ_2 da DoG:

$$\sigma^2 = \frac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 - \sigma_2^2} \ln\left(\frac{\sigma_1^2}{\sigma_2^2}\right). \quad (10)$$

Localização dos cruzamentos por zero

Após filtrar a imagem com LoG ou DoG, localize os **cruzamentos por zero**. Para cada pixel da imagem filtrada, avalie uma vizinhança 3×3 e verifique quatro pares de vizinhos opostos: vertical, horizontal e as duas diagonais. Se os sinais de um par forem opostos e a diferença absoluta entre eles exceder um limiar T , marque o pixel central como borda.

```

def cruzamentos_por_zero(img_filtrada, limiar=0):
    linhas, colunas = img_filtrada.shape
    bordas = np.zeros((linhas, colunas), dtype=np.uint8)
    for i in range(1, linhas - 1):
        for j in range(1, colunas - 1):
            pares = [
                (img_filtrada[i-1, j], img_filtrada[i+1, j]), #
                (img_filtrada[i, j-1], img_filtrada[i, j+1]), #
                (img_filtrada[i-1, j-1], img_filtrada[i+1, j+1]), #
                (img_filtrada[i-1, j+1], img_filtrada[i+1, j-1]) #
            ]
            for p1, p2 in pares:
                # <= 0 captura tambem o cruzamento exatamente em
                # zero
                if (p1 * p2 <= 0) and abs(p1 - p2) > limiar:
                    bordas[i, j] = 255
                    break
    return bordas

```

Algoritmo de Marr-Hildreth

As peças construídas até aqui — a suavização gaussiana, o Laplaciano discreto, o operador LoG e a detecção por cruzamentos por zero — reúnem-se no **algoritmo de Marr-Hildreth**. A ideia central explora o fato de que a convolução é linear e comuta com a diferenciação: aplicar o Laplaciano a uma imagem previamente suavizada por uma gaussiana equivale a convoluir a imagem com o Laplaciano da gaussiana,

$$g(x, y) = \nabla^2[G(x, y) \star f(x, y)] = [\nabla^2 G(x, y)] \star f(x, y). \quad (11)$$

A Equação (11) mostra que há dois caminhos equivalentes para o mesmo resultado: (i) suavizar a imagem com a gaussiana e, em seguida, aplicar o Laplaciano 3×3 ; ou (ii) construir o operador LoG e convoluir a imagem uma única vez, exatamente como nas subseções anteriores.

Gonzalez & Woods resumem o algoritmo em três passos:

1. Filtre a imagem de entrada com um *kernel* gaussiano passa-baixa $n \times n$, obtido por amostragem da gaussiana 2D da Equação (6).
2. Calcule o Laplaciano da imagem resultante do Passo 1 usando, por exemplo, o *kernel* 3×3 correspondente à Equação (2). [Os Passos 1 e 2 implementam a Equação (11).]
3. Localize os cruzamentos por zero da imagem resultante do Passo 2.

Cada passo corresponde a um trecho já trabalhado neste roteiro: a gaussiana passa-baixa amostra a função $G(x, y)$ da Equação (6); o Laplaciano 3×3 é a aproximação por diferenças centrais da Equação (2); e os cruzamentos por zero são obtidos com a função **cruzamentos_por_zero** já definida.

O *kernel* gaussiano do Passo 1 é normalizado para soma unitária, de modo a preservar a intensidade média da imagem (caráter passa-baixa):

```

def gaussiana(k, s):
    n = 2*k + 1
    filtro = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            r2 = (i-k)**2 + (j-k)**2
            filtro[i, j] = np.exp(-r2/(2.*s**2)) / (2*pi*s**2)
    filtro = filtro / filtro.sum() # soma 1: filtro passa-baixa
    return filtro

```

Os três passos podem ser encadeados na sequência abaixo, reaproveitando a função de cruzamentos por zero. O Laplaciano 3×3 do Passo 2 é exatamente a máscara correspondente à Equação (2).

```

from scipy.ndimage import convolve

def marr_hildreth(img, k, s, limiar=0):
    # Passo 1: suavizacao com kernel gaussiano n x n
    g = gaussiana(k, s)
    suave = convolve(img.astype(float), g, mode='nearest')
    # Passo 2: Laplaciano 3x3 (2a derivada discreta)
    L = np.array([[0, 1, 0],
                  [1, -4, 1],
                  [0, 1, 0]], dtype=float)
    resp = convolve(suave, L, mode='nearest')
    # Passo 3: cruzamentos por zero
    bordas = cruzamentos_por_zero(resp, limiar)
    return resp, bordas

```

Observação. O tamanho n do *kernel* gaussiano costuma ser o menor inteiro ímpar maior ou igual a 6σ , o que captura praticamente todo o volume da gaussiana. A rota de dois passos e a rota com o operador LoG direto não produzem respostas numericamente idênticas — por causa da normalização, do arredondamento dos coeficientes e do truncamento do suporte —, mas devem concordar na *posição* dos cruzamentos por zero, que é o que importa para a localização das bordas.

Experimentos

Realize os experimentos na sequência abaixo.

1. Construa a imagem sintética com fundo preto, um quadrado claro e um círculo claro.
2. Aplique a máscara **M** de Gonzalez & Woods.
3. Aplique a máscara **−M**.

4. Compare as respostas e os cruzamentos por zero obtidos nos itens 2 e 3.
5. Use filtros 5×5 , isto é, $k = 2$, para comparar com o tamanho da máscara de Gonzalez & Woods.
6. Fixe $\sigma_2 = 1,0$ e teste as razões $\sigma_1:\sigma_2 = 1,6:1$ e $\sigma_1:\sigma_2 = 1,75:1$.
7. Para cada razão, calcule o σ equivalente da LoG pela Equação (10).
8. Aplique o LoG amostrado e a DoG à imagem sintética.
9. Localize os cruzamentos por zero em todas as respostas filtradas.
10. Repita os testes com suporte maior, por exemplo 11×11 ($k = 5$), e compare com os resultados obtidos para 5×5 .
11. Implemente o algoritmo de Marr-Hildreth pela rota de dois passos: suavize a imagem sintética com o *kernel* gaussiano (Passo 1), aplique o Laplaciano 3×3 (Passo 2) e localize os cruzamentos por zero (Passo 3).
12. Compare a resposta da rota de dois passos com a obtida ao convoluir a imagem diretamente com o operador LoG amostrado, verificando empiricamente a equivalência da Equação (11).

Entregáveis

Ao final, apresente em seu Notebook Colab:

1. a imagem sintética usada no teste;
2. a resposta obtida com a máscara 5×5 de Gonzalez & Woods;
3. a resposta obtida com a máscara 5×5 multiplicada por -1 ;
4. os mapas de cruzamentos por zero correspondentes;
5. as respostas obtidas com LoG amostrado e DoG para as razões 1,6:1 e 1,75:1;
6. os resultados para filtros 5×5 e 11×11 ;
7. a resposta e o mapa de cruzamentos por zero produzidos pelo algoritmo de Marr-Hildreth pela rota de dois passos;
8. uma comparação entre a rota de dois passos e a aplicação direta do operador LoG, destacando a equivalência da Equação (11) e a coincidência dos cruzamentos por zero;
9. uma discussão crítica sobre convenção de sinal, coincidência dos cruzamentos por zero, tamanho do suporte, razão entre desvios padrão, arredondamento dos coeficientes, soma do filtro e a equivalência entre suavizar-e-derivar e o operador LoG.

Comente explicitamente se a inversão de sinal preservou os cruzamentos por zero, se a DoG se comportou como boa aproximação do LoG nos casos testados e se a máscara inteira de Gonzalez & Woods parece ser uma amostragem direta da função contínua ou uma aproximação discreta compacta ajustada para uso prático. Comente também se as duas rotas do algoritmo de Marr-Hildreth — suavizar-e-derivar e operador LoG direto — produziram os mesmos cruzamentos por zero.

Submissão

Prepare uma apresentação estilo PowerPoint — incluindo o algoritmo de Marr-Hildreth e a correspondência entre seus três passos e as peças construídas ao longo do roteiro — e submeta no Google Classroom junto com o *notebook* Colab contendo os experimentos acima, anotações explicativas e as respostas ao que foi arguido. Prepare-se para apresentar caso seja sorteado(a).